

AUDITORÍA TÉCNICA INTEGRAL

Proyecto: Iglesia del Nazareno Bagua

Plataforma Web Administrativa — Desarrollo de Aplicaciones Web II

Versión: commit f7b475b (HEAD) | Stack: PHP 8.x + MySQL + PDO + Composer + Apache

Patrón declarado: MVC con DAO | LOC auditadas: ~3,500 PHP

Fecha: Mayo 2026 | Marco: Rúbrica P2.1 — Unidad 2 (5° ciclo)

1. RESUMEN EJECUTIVO

El proyecto **funciona** y demuestra esfuerzo en separar responsabilidades, pero hoy está **lejos del nivel "Avanzado (4)"** de la rúbrica. El equipo declara una arquitectura MVC con DAO, pero la implementación tiene varios incumplimientos que deben corregirse antes de la entrega.

Estado actual vs. brecha hacia nivel Avanzado

Aspecto	Estado actual	Brecha
Estilo arquitectónico	MVC con DAO, parcialmente respetado	Falta documentación, rutas no amigables, sin seguridad perimetral
Capa de datos	DAOs + PDO con prepared statements	Sin Query Builder ni BaseDAO (exigido en nivel 4)
Organización	Namespaces + autoload custom	No sigue PSR-4 (exigido por rúbrica)
Verificación funcional	CRUDs sin API REST	Sin endpoints REST verbo-correctos ni colección Postman
Seguridad	Sesiones básicas + password_hash	9 vulnerabilidades de severidad alta o crítica

Puntaje estimado

Criterio	Hoy	Con remediación
Definición del Estilo Arquitectónico	2 — En proceso	4 — Avanzado
Fundamentación y Justificación	1 — Inicio	4 — Avanzado
Arquitectura de Capa de Datos	2 — En proceso	4 — Avanzado
Organización y Estándares (PSR-4)	2 — En proceso	4 — Avanzado
Verificación de Funcionalidad	2 — En proceso	4 — Avanzado
TOTAL	9/20 = 45%	20/20 = 100%

Los 3 problemas más urgentes

■ SEC-01 — Carpeta aplicacion/ accesible públicamente

El único .htaccess está en la raíz y no bloquea acceso directo a aplicacion/vistas/admin/contenidos/*.php. Cualquiera puede saltarse el login navegando a la URL directa. Esto rompe completamente la autenticación.

■ SEC-02 — Subida de archivos sin validación

NoticiaController y RecursoController no validan tipo ni extensión. Permite subir un shell.php disfrazado y ejecutarlo. Es Remote Code Execution.

■ QC-PSR4 — Autoload no es PSR-4

El autoload mapea namespaces a rutas físicas por coincidencia. La rúbrica lo exige explícitamente para nivel Avanzado (4). En Linux esto rompe por case-sensitivity.

2. MAPEO FUNCIONAL DEL SISTEMA

La plataforma tiene dos audiencias: el **sitio público (/web/)** con portada, transmisiones en vivo (Pusher), noticias, recursos y carta de salvación; y el **panel admin (/admin/)** con CRUD completo sobre 8 módulos.

Módulo	Controlador	DAO	Modelo
Autenticación	procesos/auth/*	userDAO	userLogin
Usuarios admin	RegistroController	userDAO	userLogin
Membresía	MiembroController	MiembroDAO	Miembro, Cargo
Discipulado	DiscipuladoController, GrupoController	DiscipuladoDAO, GrupoDAO	DiscipuladoModelo
Visitas	VisitaController	VisitaDAO	VisitaModelo
Noticias	NoticiaController	NoticiaDAO	(sin modelo)
Recursos	RecursoController	RecursoDAO	Recurso
Transmisiones	TransmisionController	TransmisionDAO	TransmisionModelo

3. ARQUITECTURA ACTUAL: DIAGNÓSTICO

4 desviaciones críticas detectadas

■ Desviación 1: Vistas que actúan como controladores

aplicacion/vistas/admin/contenidos/usuarios_admin.php procesa POST, instancia DAOs y decide redirecciones directamente. Una vista no debe hacer ninguna de esas cosas. Esto mezcla V con C — lo que la rúbrica llama separación de responsabilidades confusa (nivel 2).

■ Desviación 2: Múltiples puntos de entrada

Se detectaron al menos 6 puntos de entrada independientes: public/index.php, procesar_login.php, logout.php, procesar_registro.php (roto), procesar_eliminar.php y get_live_status.php. Un MVC real tiene un único Front Controller.

■ Desviación 3: Rutas no son verdaderamente amigables

El .htaccess reescribe a index.php?vista=... pero las URLs internas siguen usando query strings clásicas (index.php?vista=dashboard&seccion=membresia). Además, RegistroController expone la ruta interna /IglesiaDelNazarenoBagua/aplicacion/vistas/...

■ Desviación 4: Autoload por coincidencia, no PSR-4

Autoload.php usa str_replace('\\', '/', \$clase).''.php' que funciona porque los namespaces coinciden con carpetas. PSR-4 exige mapeo explícito de namespace prefix a base directory. En Linux la diferencia UserDAO vs userDao rompe el autoload.

4. EVALUACIÓN CONTRA LA RÚBRICA P2.1

4.1 Definición del Estilo Arquitectónico → 2/4 — En proceso

- Separación en carpetas controladores/dao/modelos/vistas.
- Las vistas procesan POST — confunde responsabilidades.
- Rutas semi-amigables solo en la home.
- Sin seguridad perimetral: aplicacion/ accesible directamente.
- Sin documentación arquitectónica (no hay README, ARCHITECTURE.md).

4.2 Fundamentación Técnica → 1/4 — Inicio

- No hay ADR (Architecture Decision Records) en el proyecto.
- No hay justificación de por qué MVC y no arquitectura hexagonal, etc.
- Solución: redactar ADR-001 comparando al menos 3 alternativas.

4.3 Arquitectura de la Capa de Datos → 2/4 — En proceso

- DAOs separados por entidad, PDO con prepared statements.
- Sin Query Builder encadenable ni BaseDAO con métodos all(), find(), where().
- listar() se reescribe en cada DAO — sin reutilización.
- Algunas queries usan query() sin escape — patrón peligroso.

4.4 Organización del Código (PSR-4) → 2/4 — En proceso

- Usa namespaces y autoloader.
- Autoloader no es PSR-4 — es carga por coincidencia.
- Inconsistencias: userLogin vs Miembro, userDAO vs MiembroDAO.
- Composer ya está instalado (Pusher), pero no se usa para autoloader.

4.5 Verificación de Funcionalidad → 2/4 — En proceso

- El sistema devuelve HTML, no JSON — no es una API REST.
- Usa GET para acciones destructivas (?eliminar=42) — viola REST y expone a CSRF.
- No hay colección Postman.
- Códigos HTTP siempre son 200, incluso en error.

5. HALLAZGOS DE SEGURIDAD

■ SEC-01 — Authentication Bypass (CRÍTICO)

Archivo: aplicacion/vistas/admin/contenidos/*.php

Problema: Solo existe .htaccess en la raíz. Cualquiera puede acceder directamente a membresia.php, noticias.php, etc. sin pasar por el login, viendo datos de miembros con direcciones, teléfonos y coordenadas GPS, y disparando acciones de mutación.

Remediación: Agregar .htaccess en aplicacion/ con 'Require all denied'. Mover contenido fuera del DocumentRoot.

■ SEC-02 — Remote Code Execution vía upload (CRÍTICO)

Archivo: NoticiaController.php, RecursoController.php

Problema: No se valida extensión ni MIME real. El nombre original se conserva. Si se sube un archivo shell.php, es accesible vía web y ejecutable. Game over.

Remediación: Lista blanca de extensiones, validación con finfo_file(), renombrado con hash aleatorio, .htaccess en uploads/ que bloquee ejecución PHP.

■ SEC-03 — Sin autenticación en endpoints de mutación (CRÍTICO)

Archivo: public/index.php (líneas 42-66), procesar_eliminar.php

Problema: Endpoints como /admin/guardarVisita y /admin/eliminarVisita reciben POST/GET y mutan la BD sin verificar sesión. Cualquiera con un curl puede crear, editar o eliminar visitas.

Remediación: Middleware de autenticación obligatorio antes de invocar cualquier controlador.

■ SEC-04 — SQL Injection latente (ALTO)

Archivo: MiembroDAO.php, NoticiaDAO.php, InicioDAO.php

Problema: Uso de query() sin parámetros. Hoy son seguros porque no reciben input externo, pero el patrón está normalizado. Un mantenedor puede agregar una variable concatenada y abrir una inyección SQL.

Remediación: Política estricta: todo va por prepared statement. Añadir PHPStan con regla SQL.

■ SEC-05 — Sin protección CSRF (ALTO)

Archivo: Todos los formularios admin

Problema: Ningún formulario tiene token CSRF. Un atacante puede engañar a un admin para que su navegador envíe un POST malicioso.

Remediación: Token CSRF por sesión generado con random_bytes(32), validado en cada POST.

■ SEC-06 — Sesiones sin endurecer (ALTO)

Archivo: procesar_login.php, dashboard.php

Problema: No se configuran cookies con HttpOnly, Secure ni SameSite. Sin timeout absoluto. Además \$_SESSION['usuario_id'] nunca se setea en login (solo \$_SESSION['usuario']), por lo que todas las visitas se registran como del usuario #1.

Remediación: Centralizar en Session::init() con cookie_params completos y timeout de 60 min.

■ SEC-07 — display_errors=1 en producción (ALTO)

Archivo: public/index.php, procesar_eliminar.php

Problema: Errores PHP exponen rutas absolutas, nombres de BD y queries SQL. Información valiosa para un atacante.

Remediación: display_errors=0, log_errors=1. Centralizar config en bootstrap.php con switch dev/prod.

■ SEC-08 — Credenciales hardcodeadas (ALTO)

Archivo: aplicacion/config/Conexion.php

Problema: user='root', pass="" hardcodeados en el código fuente. Si el proyecto está en git público, las credenciales son visibles.

Remediación: Migrar a .env con vlucas/phpdotenv.

6. HALLAZGOS DE CALIDAD DE CÓDIGO

QC-01	Inconsistencia mayúsculas/minúsculas — class UserDao declarado en userDao.php pero instanciado como new userDao(). class userLogin viola PascalCase. En Linux es case-sensitive y rompe el autoload.
QC-02	Llave extra que rompe la lógica — dashboard.php línea 94 tiene un } huérfano. Actualmente compila pero la indentación miente y confundirá a futuros mantenedores.
QC-03	procesar_registro.php está roto — La ruta del require es incorrecta desde su ubicación. La funcionalidad real ya está en usuarios_admin.php. Es código muerto y debe eliminarse.
QC-04	procesar_eliminar.php con use incorrecto — Usa dao\NoticiaDAO en lugar de aplicacion\dao\NoticiaDAO. No ejecuta porque nunca se llama, pero es accesible por URL directa. Eliminar.
QC-05	Magic numbers para roles — case 1, case 2 y in_array(\$_SESSION['rol_id'], [1, 2]) dispersos. Si un rol cambia de ID en BD, hay que cazarlo por todo el código. Solución: usar constantes Rol::ADMIN, Rol::PASTOR.
QC-06	Naked catch — errores silenciosos — RegistroController captura Exception y devuelve false sin loggear. VisitaDAO::obtenerMesesLimite() devuelve 6 como fallback silencioso. Imposible depurar.
QC-07	Lógica de presentación en DAO — VisitaDAO::listarConDetalles() agrega clase_css, icono y estado_texto al resultado. El DAO debe devolver datos puros; la presentación pertenece al controlador o a un Presenter.
QC-08	SRP roto en NoticiaController — guardarNoticia() tiene 86 líneas que mezclan validación, manejo de archivos, redirección, inserción y actualización. Debe partirse en UploadService + NoticiaService + Controller.

7. PLAN DE REMEDIACIÓN PRIORIZADO

■ Fase 1 — CRÍTICOS (antes de presentar) — ~3-4 horas

#	Tarea	Esfuerzo	Impacto
1	Crear .htaccess en aplicacion/ con Deny from all	5 min	Seguridad perimetral → Crit. 1 = 4
2	Mover credenciales BD a .env	30 min	Cierra SEC-08
3	Activar autoloader PSR-4 vía Composer	20 min	Crit. 4 = 4
4	Validar y endurecer uploads	1 h	Cierra SEC-02
5	Verificar sesión en endpoints del index.php	30 min	Cierra SEC-03
6	Apagar display_errors y centralizar bootstrap	15 min	Cierra SEC-07
7	Borrar archivos muertos (procesar_registro.php, procesar_eliminar.php)	5 min	Limpieza
8	Normalizar capitalización de clases	20 min	Crit. 4 + evita romper en Linux

■ Fase 2 — Subir a Avanzado (rúbrica = 4) — ~20 horas

#	Tarea	Esfuerzo	Impacto
10	Implementar BaseDAO + QueryBuilder encadenable	4 h	Crit. 3 = 4
11	Diseñar API REST /api/v1/... con verbos HTTP correctos	6 h	Crit. 5 = 4
12	Middleware de autenticación + tokens CSRF	2 h	Cierra SEC-05/06
13	Router de rutas amigables reales (/admin/usuarios/123)	3 h	Crit. 1 + 5
14	Redactar README.md, ARCHITECTURE.md y ADR-001	2 h	Crit. 1 + 2 = 4
15	Colección Postman con CRUD completo de los 8 módulos	2 h	Crit. 5 = 4

■ Fase 3 — Excelencia (opcional)

- Tests unitarios con PHPUnit en DAOs y QueryBuilder.
- Logger PSR-3 (Monolog) en lugar de error_log().
- Manejo de excepciones con clase HttpException jerárquica.
- Migraciones de BD con carpeta migrations/ con SQL versionado.
- GitHub Actions con linter (PHP_CodeSniffer) + análisis estático (PHPStan).

8. IMPLEMENTACIONES CONCRETAS

8.1 — Activar PSR-4 con Composer

Reemplaza composer.json para activar el autoload PSR-4 real. La carpeta aplicacion/ se renombra a src/ y los namespaces pasan de aplicacion\controladores\X a App\Controllers\X.

```
// composer.json

"autoload": {

"psr-4": { "App\\": "src/" }

},

"require": {

"php": ">=8.1",

"pusher/pusher-php-server": "^7.2",

"vlucas/phpdotenv": "^5.6"

}
```

```
# Ejecutar tras editar composer.json:

composer dump-autoload -o
```

El archivo .env en la raíz (fuera de git):

```
DB_HOST=localhost

DB_NAME=iglesiadelnazareno

DB_USER=root

DB_PASS=

APP_ENV=production

APP_DEBUG=false
```

8.2 — QueryBuilder encadenable (cumple Crit. 3 nivel 4)

El QueryBuilder permite construir queries de forma fluida sin escribir SQL crudo. Todos los valores se parametrizan automáticamente — sin posibilidad de SQL Injection. Un DAO concreto que antes tenía 60 líneas de SQL queda así:

```
// Antes (SQL crudo en cada DAO):

$stmt = $this->pdo->prepare(

"SELECT * FROM miembros WHERE estado=? ORDER BY apellidos ASC"

);

$stmt->execute([1]);
```

```
// Después (QueryBuilder):

public function listarActivos(): array
{
    return $this->where("estado", "=", 1)
        ->orderBy("apellidos", "ASC")
        ->all();
}
```

Métodos disponibles en el QueryBuilder:

Método	Descripción
where(\$col, \$op, \$val)	Agrega condición WHERE parametrizada
orderBy(\$col, \$dir)	Agrega ORDER BY con dirección ASC/DESC
limit(\$n) / offset(\$n)	Paginación
join(\$tabla, \$cola, \$colB)	INNER/LEFT/RIGHT JOIN
all()	Devuelve array de todos los resultados
find(\$id)	Devuelve fila por PK o null
first()	Devuelve primera fila o null
insert(\$datos)	INSERT – devuelve id creado
update(\$datos)	UPDATE – requiere where() previo
delete()	DELETE – requiere where() previo
count()	Cuenta registros

8.3 — Upload seguro (cierra SEC-02)

UploadService valida MIME real con `finfo_file()`, aplica lista blanca, renombra el archivo con hash aleatorio y aplica permisos restrictivos.

```
// src/Services/UploadService.php

private const MIME_PERMITIDOS = [

    "image/jpeg" => "jpg",

    "image/png" => "png",

    "image/webp" => "webp",

    "application/pdf" => "pdf",

];

$finfo = new \finfo(FILEINFO_MIME_TYPE);

$mimeTypeReal = $finfo->file($archivo["tmp_name"]);
```

```

if (!isset(self::MIME_PERMITIDOS[$mimeReal])) {

    throw new \RuntimeException("Tipo no permitido: $mimeReal");

}

$nombreSeguro = bin2hex(random_bytes(16)) . "." . $extension;

move_uploaded_file($archivo["tmp_name"], $destino . $nombreSeguro);

chmod($destino . $nombreSeguro, 0644);

```

Además, crear en uploads/ un .htaccess que bloquee ejecución PHP:

```
Require all denied
```

8.4 — API REST con verbos HTTP correctos (cierra Crit. 5)

El router mapea rutas con parámetros dinámicos ({id}) a métodos del controlador usando el verbo HTTP correcto. Soporta method override por POST hidden field para formularios HTML que no admiten PUT/DELETE.

```

// src/routes.php – ejemplo módulo Miembros

$router->get ("/api/v1/miembros", [MiembroController, "index"], [$auth]);

$router->get ("/api/v1/miembros/{id}", [MiembroController, "show"], [$auth]);

$router->post ("/api/v1/miembros", [MiembroController, "store"], [$auth,$csrf]);

$router->put ("/api/v1/miembros/{id}", [MiembroController, "update"], [$auth,$csrf]);

$router->delete("/api/v1/miembros/{id}", [MiembroController, "destroy"], [$auth,$csrf]);

```

Códigos HTTP que devuelve el controlador según el caso:

Código	Cuándo usarlo
200 OK	GET exitoso, PUT/PATCH exitoso
201 Created	POST exitoso — incluye id creado en body
204 No Content	DELETE exitoso — sin body
400 Bad Request	Payload malformado (JSON inválido)
401 Unauthorized	Sin sesión activa
403 Forbidden	Sesión activa pero sin permiso (rol)
404 Not Found	Recurso no existe
422 Unprocessable Entity	Validación fallida (campo requerido faltante)
500 Internal Server Error	Error inesperado del servidor

9. HOJA DE RUTA PARA NIVEL AVANZADO (Rúbrica = 4)

Lista mínima para que cada criterio se evalúe en 4/4 puntos:

■ Criterio 1: Estilo arquitectónico

- docs/ARCHITECTURE.md con diagrama del sistema
- .htaccess con seguridad perimetral (deny en src/, bloqueo PHP en uploads/)
- Rutas amigables reales (/admin/miembros/15) vía Router
- Front Controller único en public/index.php

■ Criterio 2: Fundamentación técnica

- ADR-001 que compare MVC con al menos 3 alternativas
- Sección Trade-offs honesta y sustentada
- Referencias a PSR, OWASP y documentación oficial

■ Criterio 3: Capa de datos

- QueryBuilder encadenable con where(), orderBy(), limit(), join()
- BaseDAO con métodos reutilizables all(), find(), create(), update(), delete()
- Cada DAO concreto extiende BaseDAO y agrega solo métodos del dominio

■ Criterio 4: PSR-4 + organización

- composer.json con sección psr-4
- Estructura /src/{Config,Core,Controllers,Dao,Models,Services,Middleware}
- Namespaces consistentes en PascalCase — sin archivos muertos
- README con instrucciones de instalación y tablas SQL

■ Criterio 5: Verificación funcional

- API REST /api/v1/... con verbos GET/POST/PUT/DELETE
- Códigos HTTP semánticos (200, 201, 204, 400, 401, 404, 422, 500)
- Validación de payload en POST/PUT con mensajes de error claros
- Colección Postman exportada con cada endpoint demostrado

10. CONCLUSIÓN Y RECOMENDACIÓN FINAL

El proyecto es **salvable y tiene una base razonable**, pero hoy está en nivel 2 en la mayoría de criterios. Con **3-4 horas de remediación crítica** y **~20 horas de mejoras estructurales**, es completamente viable llegar a **20/20 en la rúbrica**.

Prioridades claras

→ Hoy (no negociable):

- .htaccess en aplicacion/ — bloquea authentication bypass
- Apagar display_errors — deja de exponer rutas internas
- Validar uploads — cierra la puerta al RCE

→ Esta semana:

- Composer PSR-4 — una línea en composer.json, dump-autoload, renombrar carpeta
- QueryBuilder + BaseDAO — el cambio más impactante para la rúbrica
- README + ADR-001 — documentación que sube dos criterios de golpe

→ Antes de presentar:

- API REST + colección Postman — el evaluador lo probará en vivo
- Middleware de autenticación + CSRF — seguridad perimetral real
- Rutas amigables en al menos los módulos principales

Fortalezas a preservar

- PDO con prepared statements en la mayoría de DAOs — bien hecho.
- Hash de contraseñas con bcrypt — correcto.
- Uso de transacciones en MiembroDAO::registrar() — patrón profesional.
- Separación inicial en carpetas controladores/dao/modelos — base sólida.
- Integración con Pusher para tiempo real — dependencia sana y bien acotada.

Auditoría técnica generada para fines académicos — Rúbrica P2.1, Unidad 2, 5° ciclo — Mayo 2026